

Department of Electronic and  
Telecommunication Engineering  
University of Moratuwa

**EN3150 - Pattern Recognition**



Assignment 01: Learning from data and related  
challenges and linear models for regression

210490L Prabodha K.P.K.A.

## Contents

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Data Pre-Processing</b>                                            | <b>2</b>  |
| <b>2</b> | <b>Learning from data</b>                                             | <b>4</b>  |
| 2.1      | Generate data . . . . .                                               | 4         |
| 2.2      | Run code multiple times . . . . .                                     | 4         |
| 2.3      | Why the Linear Regression Model is Different Across Instances . . . . | 4         |
| 2.4      | Increase the number of data samples to 10,000 . . . . .               | 5         |
| <b>3</b> | <b>Linear regression on real-world data</b>                           | <b>7</b>  |
| 3.1      | Load the dataset . . . . .                                            | 7         |
| 3.2      | No. of Independent variables and Dependent variables . . . . .        | 7         |
| 3.3      | Applicability of Linear Regression . . . . .                          | 7         |
| 3.4      | NaN/missing values . . . . .                                          | 7         |
| 3.5      | Select independent four other features . . . . .                      | 8         |
| 3.6      | Split data . . . . .                                                  | 8         |
| 3.7      | Train a linear regression model . . . . .                             | 9         |
| 3.8      | Highly contributes independent variable . . . . .                     | 9         |
| 3.9      | Select another set of features and Train a model again . . . . .      | 9         |
| 3.10     | Calculations . . . . .                                                | 10        |
| <b>4</b> | <b>Performance evaluation of Linear regression</b>                    | <b>12</b> |
| 4.1      | Observed the table . . . . .                                          | 12        |
| 4.2      | Compute residual standard error (RSE) . . . . .                       | 12        |
| 4.3      | Compute R-squared $R^2$ . . . . .                                     | 12        |
| 4.4      | Compare between RSE and R-squared $R^2$ . . . . .                     | 13        |
| <b>5</b> | <b>Linear Regression Impact on Outliers</b>                           | <b>14</b> |
| 5.1      | What happens when $a \rightarrow 0$ ? . . . . .                       | 14        |
| 5.2      | Minimizing the influence of outliers ( $ r_i  \geq 40$ ) . . . . .    | 14        |

# 1 Data Pre-Processing

## Observations

- **Feature 1:** The data shows that most values are close to 0, with a few large spikes (outliers).
- **Feature 2:** The data is spread out over a wide range, with a lot of variations. (Mean is approximately looks like around the zero)

## Scaling Methods

### 1. Standard Scaling:

- Formula:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

- This method centers the data around 0 and scales it using the standard deviation. It is useful when data has a normal (Gaussian) distribution or when the mean and variance need to be considered.

### 2. Min-Max Scaling:

- Formula:

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- This method scales the data into a fixed range, usually between 0 and 1, by adjusting the minimum and maximum values. It is useful when you want all features to be within the same specific range.

### 3. Max-Abs Scaling:

- Formula:

$$X_{\text{scaled}} = \frac{X}{|X_{\max}|}$$

- This method scales each feature by its maximum absolute value, ensuring the data stays between -1 and 1. It is ideal for sparse data (mostly zeros) or data centered around zero with outliers.

## Selected Scaling Methods

### For Feature 1

- Use **Max-Abs Scaling**:

$$X_{\text{scaled}} = \frac{X}{|X_{\max}|}$$

- **Justification:** Since most of the data in Feature 1 is near zero with a few large outliers, Max-Abs Scaling will preserve the nature of the data by keeping the outliers intact and scaling the values between -1 and 1 without distorting the sparse structure.

### For Feature 2

- Use **Standard Scaling**:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

- **Justification:** Feature 2 has more variability and a wider range of values. Standard Scaling will center the data around 0 and account for the variability by scaling the values based on their standard deviation. This method ensures that the overall distribution of the data is preserved.

## Conclusion

- **Feature 1** (with sparse data and outliers) is best handled with **Max-Abs Scaling** to preserve the zero-centered structure and the outliers.
- **Feature 2** (with more variability) is best scaled using **Standard Scaling** to ensure the data is centered and scaled according to its spread.

## 2 Learning from data

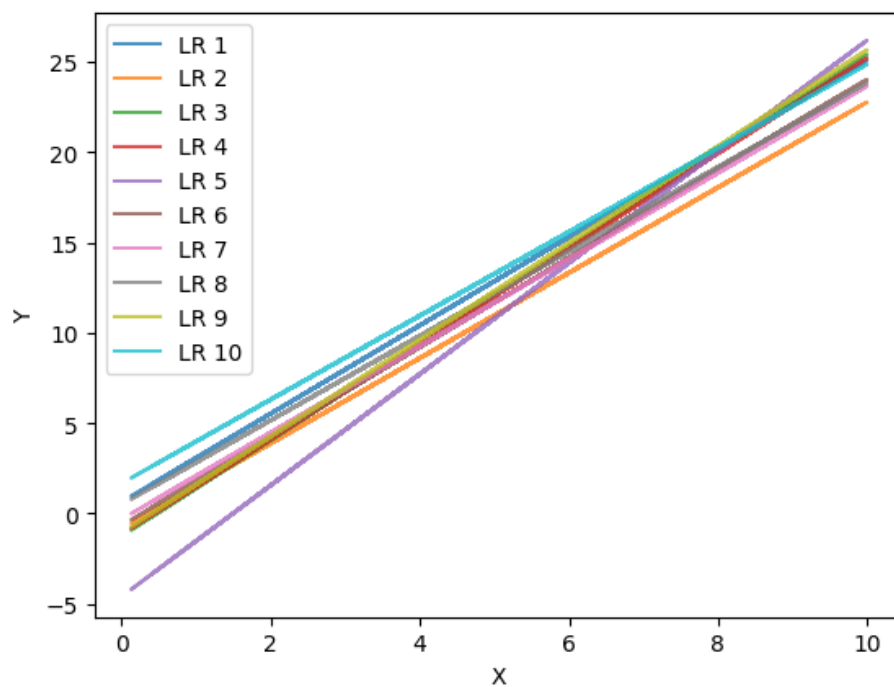
### 2.1 Generate data

Done in code

### 2.2 Run code multiple times

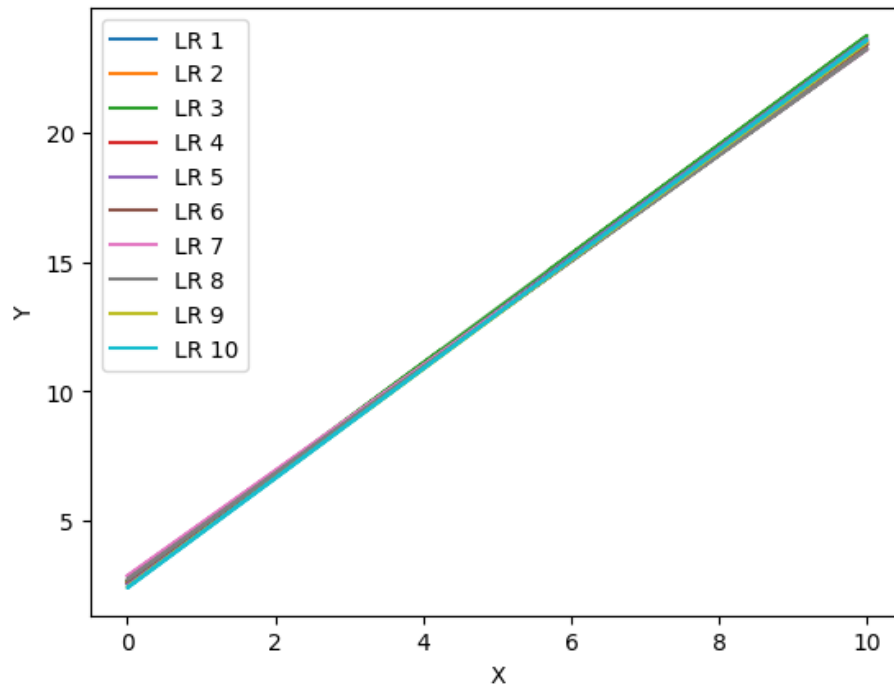
The `train_test_split` function randomly partitions the data. The `random_state` parameter ensures that this random division can be reproduced when a specific seed is utilized. However, if the seed value is generated randomly on each occasion, it will vary with every execution, leading to different configurations of the dataset in training and testing subsets.

### 2.3 Why the Linear Regression Model is Different Across Instances



When fitting the linear regression model on multiple occasions (10 instances), a new training dataset is generated each time by randomly dividing the original dataset into training and testing subsets. This random division results in different training data for each instance. The parameters learned by the model (slope and intercept) exhibit variations, indicating that the regression model experiences slight changes with each fitting.

## 2.4 Increase the number of data samples to 10,000



### Observation

1. When the number of data samples is increased to 10,000 (`n_samples = 10000`), the regression lines produced from multiple iterations exhibit a high degree of similarity.
2. The lines nearly coincide, displaying minimal variation in contrast to those generated with only 100 samples.

### Reason for Different Behavior

#### 1. Increased Sample Size:

- With 10,000 samples, the training dataset provides a more accurate representation of the overall dataset.
- This results in more consistent and dependable outcomes during the linear regression process.

#### 2. Diminished Randomness Effects:

- A larger sample size reduces the influence of random partitioning into training and testing sets.
- Then, the regression lines from various iterations appear almost indistinguishable.

### 3. Enhanced Alignment with the Actual Relationship:

- A larger sample size enables the model to more precisely identify the fundamental linear relationship between  $X$  and  $Y$ .
- The model's fit is improved, leading to predictions that are typically nearer to the actual values.

## Conclusion

Larger datasets contribute to more stable and consistent model performance, whereas smaller datasets may lead to increased variability in results due to the randomness inherent in data partitioning.

## 3 Linear regression on real-world data

### 3.1 Load the dataset

Done in code.

### 3.2 No. of Independent variables and Dependent variables

- No. of independent variables: 33
  - Categorical features: 30
  - Continuous features: 3
- No. of dependant variables: 2
  - 1. aveOralF
  - 2. aveOralM

### 3.3 Applicability of Linear Regression

The dataset includes certain categorical data. Given that linear regression necessitates numerical inputs, it is essential to transform categorical variables into numerical formats. Below are several methods to accomplish this:

1. **Label Encoding:** Assign a unique number to each category. For example, "Red" = 1, "Blue" = 2, "Green" = 3.
2. **One-Hot Encoding:** Create new binary columns for each category. For example, if you have "Red", "Blue" and "Green" you get three columns: "Red" (1 or 0), "Blue" (1 or 0), and "Green" (1 or 0).
3. **Ordinal Encoding:** If categories have a natural order (like "Low", "Medium", "High"), assign numbers based on that order, such as "Low" = 1, "Medium" = 2, "High" = 3.

### 3.4 NaN/missing values

No

`dropna()` function removes all rows with NULL values from the DataFrame. Think of this kind of scenario. If that remove 3 rows from the dependent variable and remove 5 rows from the independent variables. Then there will pop up some dispatches. The solution for that is to concatenate both independent features  $X$  and dependant variable  $Y$  together. After that operate with `dropna()` function. Then, split back these data to  $X$  and  $Y$ . Here is the correct code:



```
1 # Combine X_train and y_train to drop NaNs together
2 Data_before_clean = pd.concat([X_encoded, y_selected], axis=1)
3
4 # Drop rows with any NaN values
5 Data_after_clean = Data_before_clean.dropna()
6
7 # Separate X_train and y_train again
8 X_encoded = Data_after_clean.iloc[:, :-1]
9 y_selected = Data_after_clean.iloc[:, -1]
```

### 3.5 Select independent four other features

- Age
- T\_atm
- Humidity
- Distance
- T\_RC\_Max1

```
1 # Select 'aveOralM' as the dependent variable
2 y_selected = y['aveOralM']
3
4 # Select independent features - Age and four other features
5 selected_features = ['Age', 'T_atm', 'Humidity', 'Distance', '
    T_RC_Max1']
6
7 # Create a new DataFrame with the selected independent features
8 X_selected = X[selected_features]
9
10 # Display the selected features and the target variable
11 print("Selected Independent Features:")
12 print(X_selected.head())
13
14 print("\nSelected Dependent Feature (aveOralM):")
15 print(y_selected.head())
```

### 3.6 Split data

```
1 # Split data: 80% for training, 20% for testing
2 X_train, X_test, y_train, y_test = train_test_split(X_encoded,
    y_selected, test_size=0.2, random_state=42)
```

### 3.7 Train a linear regression model

```
1 # Initialize the Linear Regression model
2 model = LinearRegression()
3
4 # Train the model with the training data
5 model.fit(X_train, y_train)
```

### 3.8 Highly contributes independent variable

| Feature   | Coefficient |
|-----------|-------------|
| T_RC_Max1 | 0.765347    |
| Age_40-60 | 0.691407    |
| Age_41-50 | 0.243059    |
| Age_21-30 | 0.167224    |
| Age_51-60 | 0.104276    |
| Age_26-30 | 0.059082    |
| Age_21-25 | 0.047461    |
| Age_31-40 | 0.009003    |
| Distance  | 0.002541    |
| Humidity  | 0.001499    |
| T_atm     | -0.053396   |

- Highest one: T\_RC\_Max1
- Coefficient: 0.765347

### 3.9 Select another set of features and Train a model again

```
1 # I have chosen 'T_atm', 'Humidity', 'Distance', and 'T_RC_Max1' as
   additional features based on general relevance.
2 selected_features = ['T_OR1', 'T_OR_Max1', 'T_FHC_Max1', 'T_FH_Max1'
3 ]
4 # Create a new data frame with the selected independent features
5 X_selected = X[selected_features]
6
7 # Split data: 80% for training, 20% for testing
8 X_train, X_test, y_train, y_test = train_test_split(X_encoded,
9   y_selected, test_size=0.2, random_state=42)
10
11 # Initialize the Linear Regression model
12 model = LinearRegression()
13
14 # Train the model with the training data
15 model.fit(X_train, y_train)
```

```
15
16 # Get the estimated coefficients
17 coefficients = model.coef_
```

### 3.10 Calculations

```
1 import statsmodels.api as sm
2
3 # Add a constant column to the feature matrix (required by
4   statsmodels)
5 X_train_with_constant = sm.add_constant(X_train)
6
7 # Fit the OLS (Ordinary Least Squares) model
8 ols_model = sm.OLS(y_train, X_train_with_constant).fit()
9
10 # Get summary statistics of the model
11 summary = ols_model.summary()
12
13 # Print the summary
14 print(summary)
15
16 # Predict using the train set
17 y_pred = model.predict(X_train)
18
19 # Residuals
20 residuals = y_train - y_pred
21
22 # 1. Residual Sum of Squares (RSS)
23 RSS = np.sum(residuals ** 2)
24
25 # 2. Mean Squared Error (MSE)
26 MSE = mean_squared_error(y_train, y_pred)
27
28 # 3. R-squared (R2)
29 R_squared = r2_score(y_train, y_pred)
30
31 # 4. Residual Standard Error (RSE)
32 N = len(y_train) # Total number of data samples
33 d = X_train.shape[1] # Number of independent features
34 RSE = np.sqrt(RSS / (N - d - 1))
35
36 # Print the calculated metrics
37 print(f"Residual Standard Error (RSE): {RSE}")
38 print(f"Residual Sum of Squares (RSS): {RSS}")
39 print(f"Mean Squared Error (MSE): {MSE}")
40 print(f"R-squared (R2): {R_squared}")
```

1. Calculate the following:

- Residual Sum of Squares (RSS):  $RSS = 98.69733522134523$
- Residual Standard Error (RSE):  $RSE = 0.34885285890422885$

- Mean Squared Error (MSE):  $MSE = 0.12095261669282503$
- $R^2$  Statistic:  $R^2 = 0.5545367157143727$
- Standard Error, t-statistic, and p-value for each feature:

| Feature           | Coef    | Std Err | t      | P t   |
|-------------------|---------|---------|--------|-------|
| <b>const</b>      | 10.8908 | 0.840   | 12.964 | 0.000 |
| <b>T_OR1</b>      | 0.3299  | 0.074   | 4.459  | 0.000 |
| <b>T_OR_Max1</b>  | 0.1085  | 0.073   | 1.494  | 0.136 |
| <b>T_FHC_Max1</b> | 0.0857  | 0.026   | 3.239  | 0.001 |
| <b>T_FH_Max1</b>  | 0.2204  | 0.031   | 7.107  | 0.000 |

- It is observed that the p-values are significantly low. This suggests that there is substantial evidence opposing the null hypothesis. Consequently, we can reject the null hypothesis.

## 4 Performance evaluation of Linear regression

### 4.1 Observed the table

### 4.2 Compute residual standard error (RSE)

- The Residual Standard Error (RSE) is calculated using the formula:

$$\text{RSE} = \sqrt{\frac{\text{SSE}}{N - p}}$$

where:

- SSE means Sum of Squared Errors.
- $N$  is the number of data samples.
- $p$  is the number of predictors (including the intercept).

For Model A ( $p_A = 2$ ):

$$\text{RSE}_A = \sqrt{\frac{9}{10000 - 2}} \approx 0.03$$

For Model B ( $p_B = 5$ ):

$$\text{RSE}_B = \sqrt{\frac{2}{10000 - 5}} \approx 0.0142$$

- **Conclusion:** Model B has a lower RSE, so it performs better.

### 4.3 Compute R-squared $R^2$

The  $R^2$  value is calculated using the formula:

$$R^2 = 1 - \frac{\text{SSE}}{\text{TSS}}$$

For Model A:

$$R_A^2 = 1 - \frac{9}{90} = 0.9$$

For Model B:

$$R_B^2 = 1 - \frac{2}{10} = 0.8$$

- **Conclusion:** Model A has a higher  $R^2$ , so it performs better.

## 4.4 Compare between RSE and R-squared $R^2$

$R^2$  isn't always a fair way to compare models with different numbers of inputs because it tries to improve just by adding more inputs, even if they don't really help the model. On the other hand, RSE (Residual Standard Error) shows how much the model's predicted values deviate from the actual values.

- **Conclusion:** RSE is a fairer way to compare these models because it considers how complicated the model is (how many inputs it has) and gives a better idea of how accurate the model's predictions are.

## 5 Linear Regression Impact on Outliers

### 5.1 What happens when $a \rightarrow 0$ ?

**For  $L_1(w)$ :**

When  $a \rightarrow 0$ ,  $L_1(w)$  becomes 1, meaning it doesn't depend on the residuals anymore, giving a constant value.

**For  $L_2(w)$ :**

Similarly, when  $a \rightarrow 0$ ,  $L_2(w)$  also approaches 1, ignoring the residual values, becoming insensitive to the actual data.

So, both  $L_1(w)$  and  $L_2(w)$  give the same result when  $a = 0$ .

### 5.2 Minimizing the influence of outliers ( $|r_i| \geq 40$ )

To reduce the effect of outliers, we need to choose  $a$  carefully:

- When  $a = 2.5$ : Both  $L_1$  and  $L_2$  are sensitive to large residuals, meaning they still care about outliers.
- When  $a = 25$ : Both functions are less sensitive to outliers.  $L_2$  reacts more smoothly, but  $L_1$  still handles outliers effectively.
- When  $a = 100$ : Both functions ignore outliers, with  $L_1$  showing more resistance.

Thus, using  $a = 25$  with  $L_1(w)$  is better because it gradually reduces outliers' impact while keeping the function sensitive to smaller residuals.